

# LANGKAH LANGKAH MENGGUNAKAN APACHE SPARK

## Langkah 1: Membuat Direktori Proyek Tubes ABD

```
pome@FIKRIALPOME-2005: ~ X + v
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ABDILLAH FIKRI POME>wsl
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.6.87.2-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Fri May 22 11:27:42 WIB 2026

System load:  1.12               Processes:            69
Usage of /:   2.8% of 1006.85GB   Users logged in:     0
Memory usage: 4%                IPv4 address for eth0: 172.28.109.209
Swap usage:   0%

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

This message is shown once a day. To disable it please create the
pome@FIKRIALPOME-2005:/mnt/c/Users/ABDILLAH FIKRI POME$ mkdir -p ~/tubes-abd
pome@FIKRIALPOME-2005:/mnt/c/Users/ABDILLAH FIKRI POME$ cd ~/ tubes-abd
-bash: cd: too many arguments
pome@FIKRIALPOME-2005:/mnt/c/Users/ABDILLAH FIKRI POME$ cd ~/tubes-abd
pome@FIKRIALPOME-2005:~/tubes-abd$
```

pada langkah pertaman ini kita memulai dengan mengetik “WSL” terlebih dahulu, kemudian membuat data untuk mengerjakan tubes abd ini, dan kemudian masuk ke dalam file tersebut untuk mengerjakan hal selanjutnya.

## Langkah 2: Konfigurasi dan Menjalankan Layanan Docker

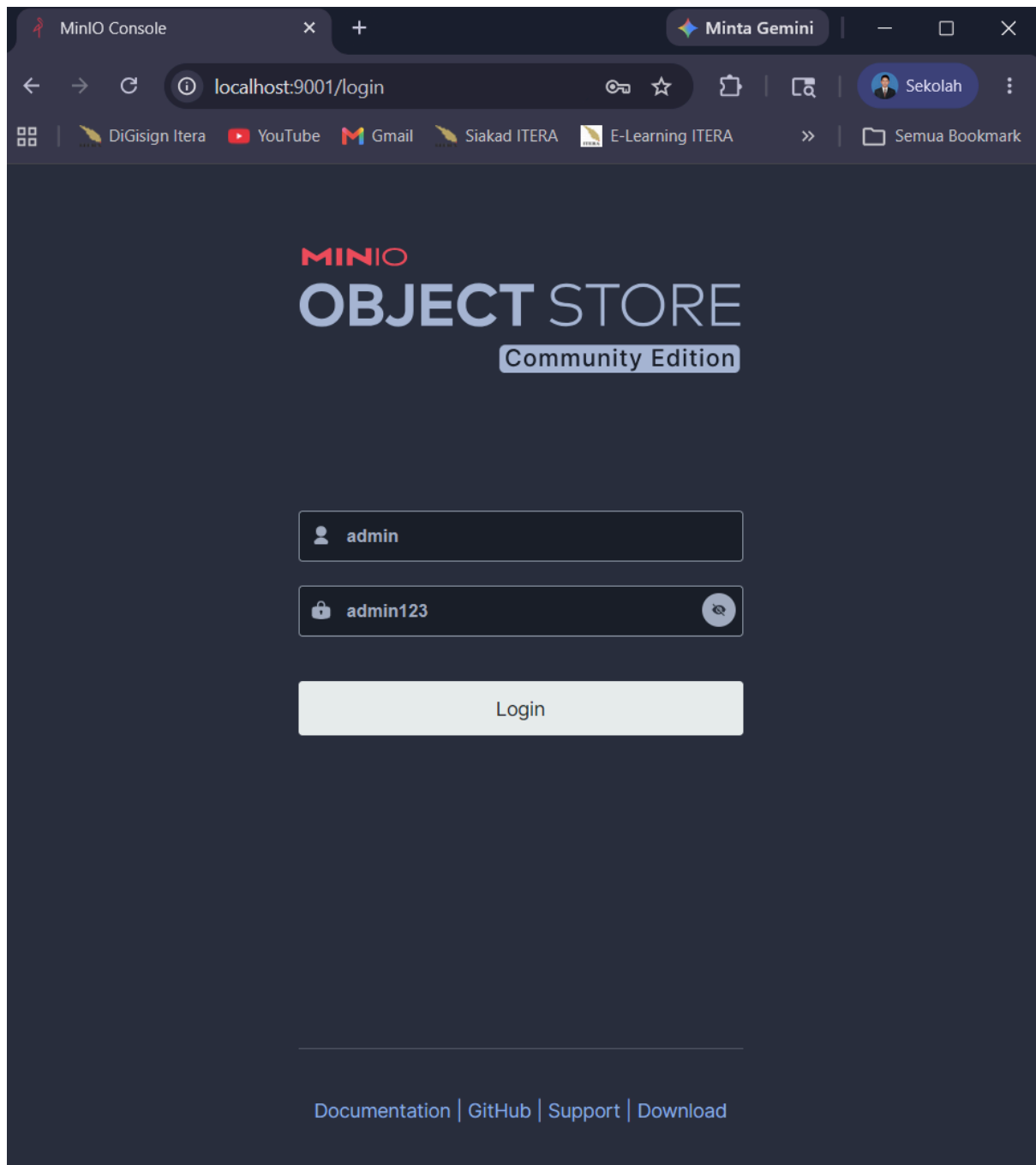
```
pome@FIKRIALPOME-2005:/mnt/c/Users/ABDILLAH_FIKRI_POME$ cd ~/tubes-abd
pome@FIKRIALPOME-2005:~/tubes-abd$
pome@FIKRIALPOME-2005:~/tubes-abd$ docker --version
Docker version 29.2.1, build a5c7197
pome@FIKRIALPOME-2005:~/tubes-abd$ docker compose version
Docker Compose version v5.0.2
pome@FIKRIALPOME-2005:~/tubes-abd$ mkdir data raw-data
mkdir: cannot create directory 'data': File exists
pome@FIKRIALPOME-2005:~/tubes-abd$ nano docker-compose.yml
pome@FIKRIALPOME-2005:~/tubes-abd$ docker compose up -d
WARN[0000] /home/pome/tubes-abd/docker-compose.yml: the attribute 'version'
is obsolete, it will be ignored, please remove it to avoid potential confusi
on
[+] up 12/12
  ✓Image minio/minio:latest Pulled 22.0s
  ✓Network tubes-abd_default Created 0.9s
  ✓Container bigdata-minio Created 0.8s
pome@FIKRIALPOME-2005:~/tubes-abd$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED
STATUS        PORTS
NAMES
ebb33bf63a31   minio/minio:latest                 "/usr/bin/docker-ent...  13 seconds ago
Up 12 seconds  0.0.0.0:9000-9001->9000-9001/tcp, [::]:9000-9001->9000-900
1/tcp         bigdata-minio
pome@FIKRIALPOME-2005:~/tubes-abd$ |
```

```
pome@FIKRIALPOME-2005: ~ × + v
GNU nano 6.2 docker-compose.yml
version: '3.8'
services:
  minio:
    image: minio/minio:latest
    container_name: bigdata-minio
    ports:
      - "9000:9000"
      - "9001:9001"
    environment:
      MINIO_ROOT_USER: admin
      MINIO_ROOT_PASSWORD: admin123
    volumes:
      - ./data:/data
    command: server /data --console-address ":9001"
```

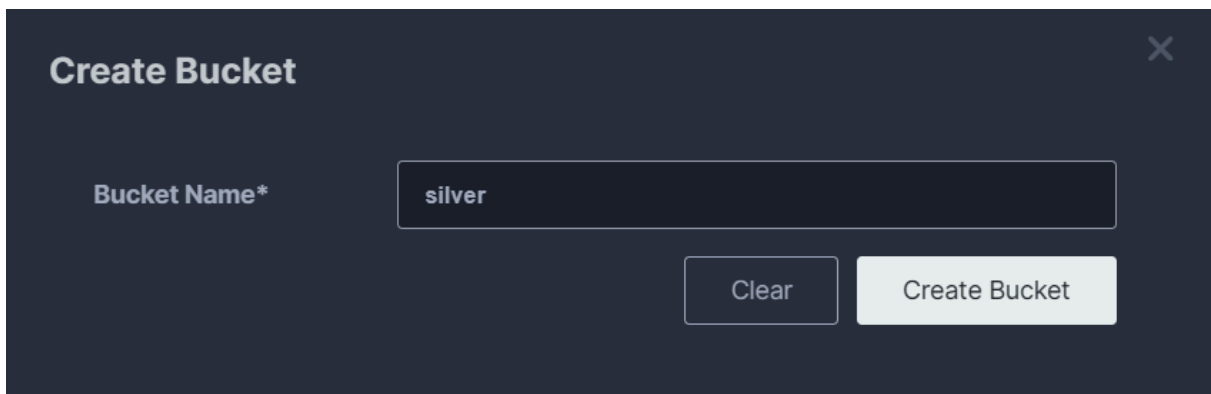
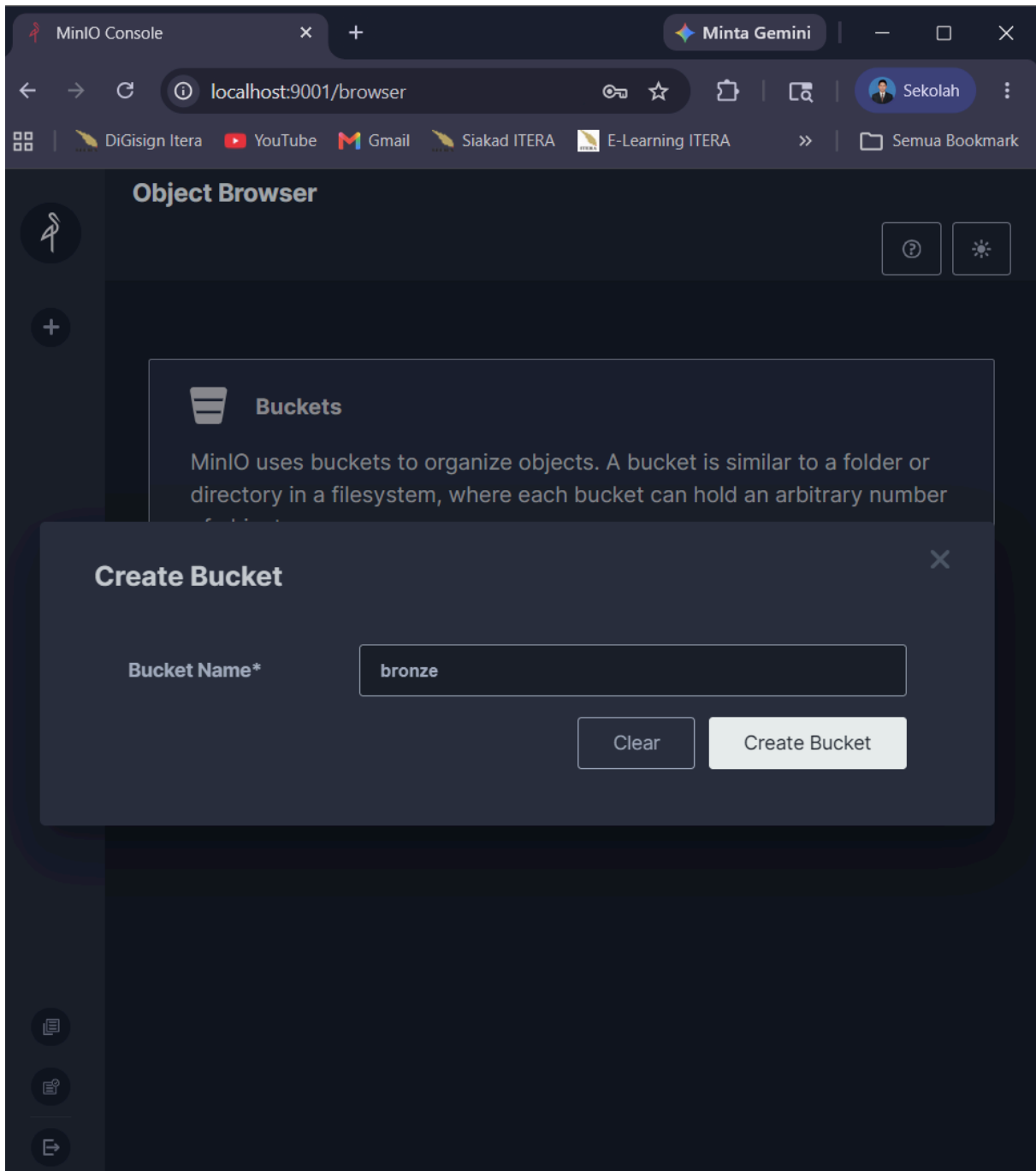
Pada langkah ini, kami membuat file konfigurasi docker-compose.yml menggunakan nano yang mendefinisikan dua layanan utama: MinIO sebagai object storage (berjalan di port 9000 dan 9001) dengan kredensial admin/admin123, serta Apache Spark menggunakan image

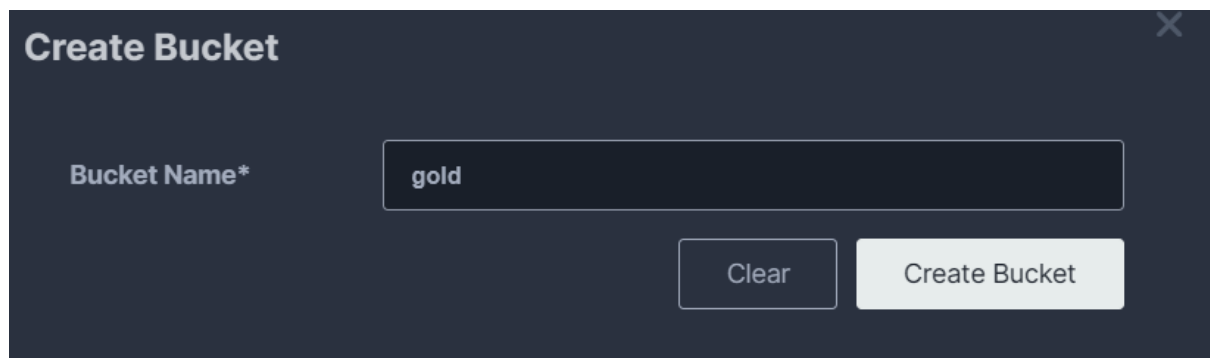
jupyter/pyspark-notebook (port 8888 untuk Jupyter Notebook dan 4040 untuk Spark UI). Setelah konfigurasi selesai, kami menjalankan perintah docker compose up -d untuk menjalankan seluruh layanan secara background, dan memverifikasi bahwa container bigdata-minio dan bigdata-spark berhasil berjalan.

### Langkah 3: Login ke MinIO Console dan Membuat Bucket



<http://localhost:9001/browser/gold>



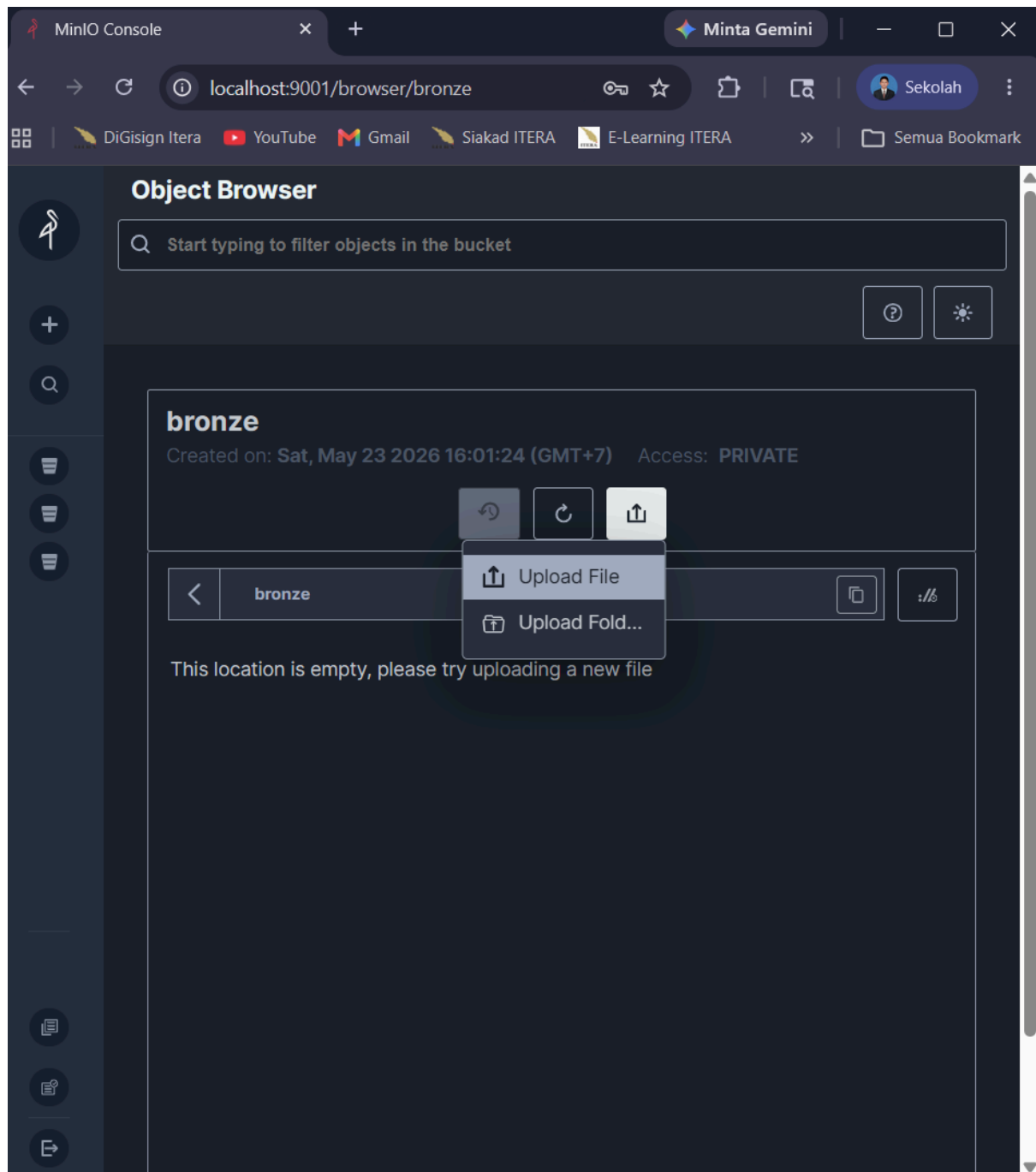


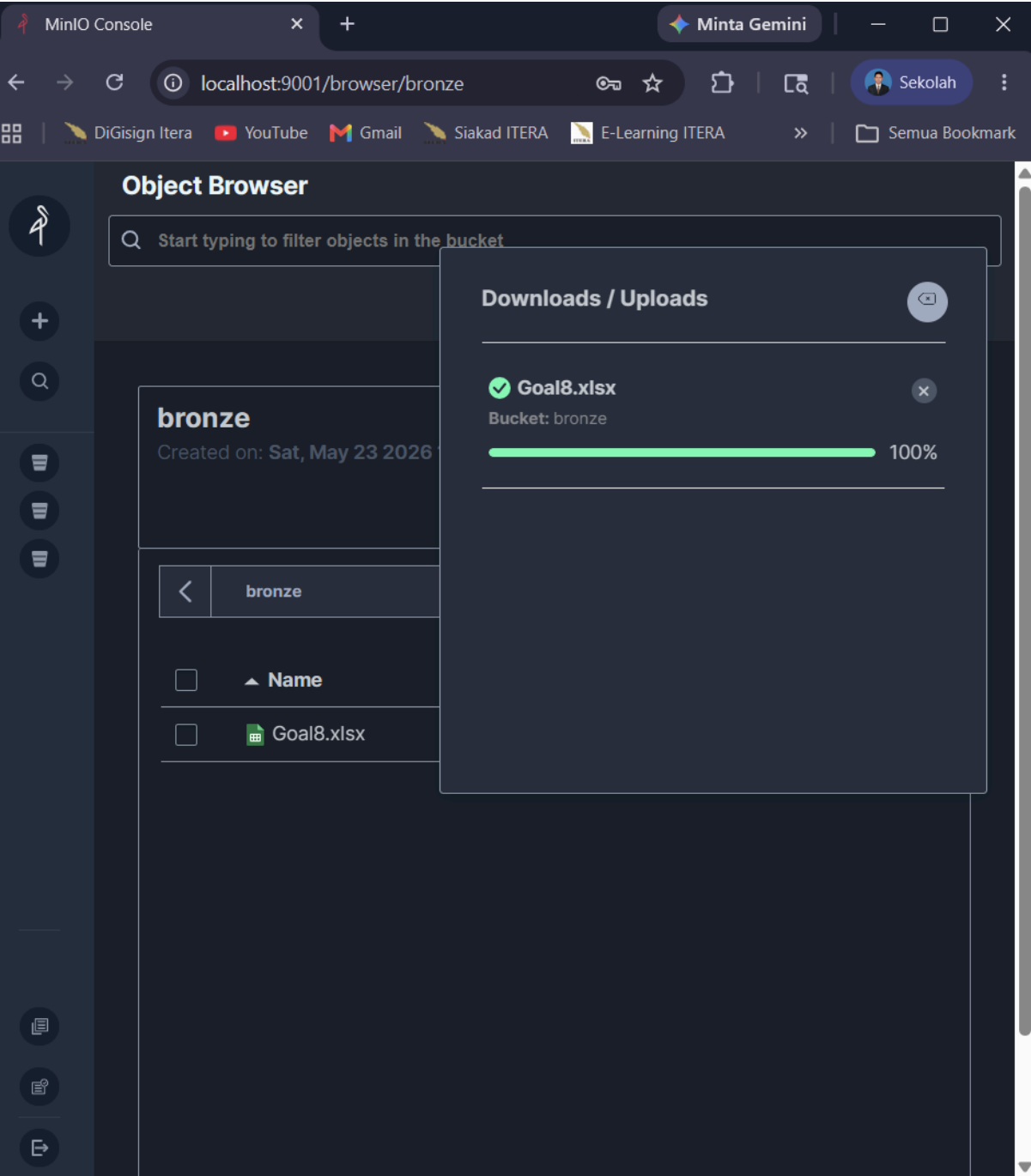
**Create Bucket**

**Bucket Name\***

Kami mengakses MinIO Console melalui browser di alamat <http://localhost:9001> menggunakan username admin dan password admin123. Setelah berhasil login, kami membuat tiga bucket yang mewakili lapisan Arsitektur Medallion, yaitu bronze sebagai penyimpanan data mentah, silver untuk data yang telah dibersihkan, dan gold untuk data agregasi hasil analitik.

## Langkah 4: Mengunggah Data Mentah ke Bronze Layer





```
pome@FIKRIALPOME-2005: ~ × + ▾
GNU nano 6.2 docker-compose.yml
services:
  minio:
    image: minio/minio:latest
    container_name: bigdata-minio
    ports:
      - "9000:9000"
      - "9001:9001"
    environment:
      MINIO_ROOT_USER: admin
      MINIO_ROOT_PASSWORD: admin123
    volumes:
      - ./data:/data
    command: server /data --console-address ":9001"

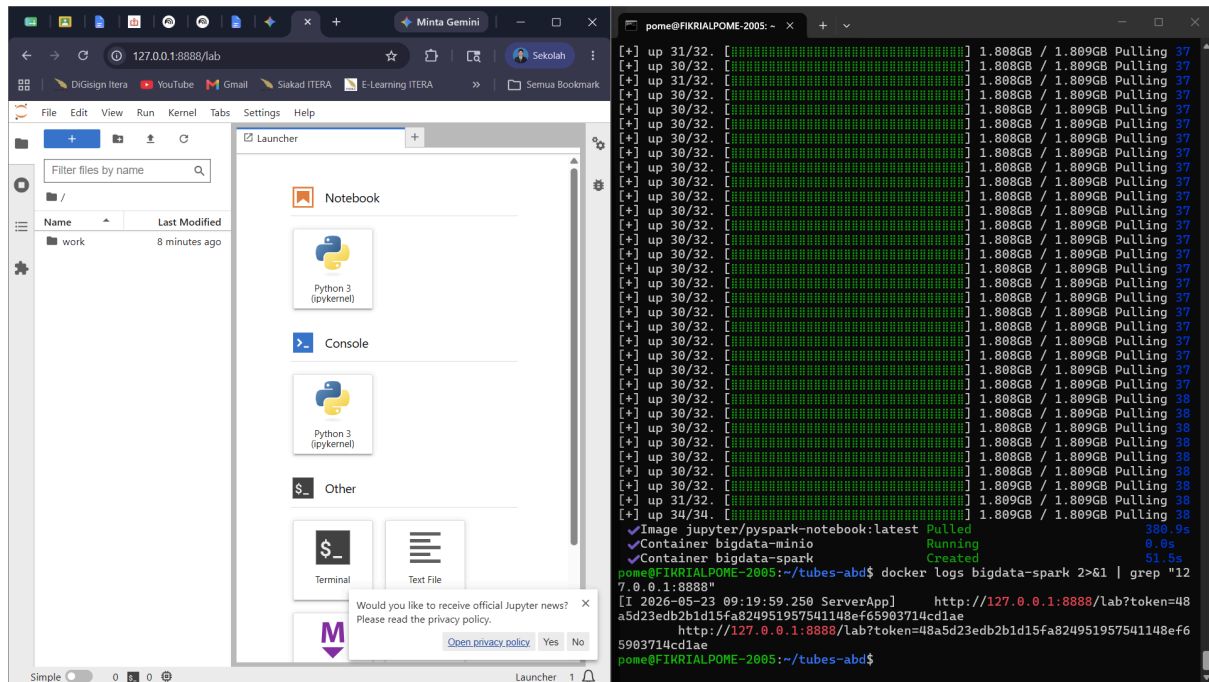
  spark:
    image: jupyter/pyspark-notebook:latest
    container_name: bigdata-spark
    ports:
      - "8888:8888"    # Port untuk Jupyter Notebook
      - "4040:4040"    # Port untuk Spark UI
    environment:
      JUPYTER_ENABLE_LAB: "yes"
    volumes:
      - ./app:/home/jovyan/work
    depends_on:
      - minio
```



```
pome@FIKRIALPOME-2005: ~ × + ▾
pome@FIKRIALPOME-2005:~/tubes-abd$ nano docker-compose.yml
pome@FIKRIALPOME-2005:~/tubes-abd$ mkdir app
pome@FIKRIALPOME-2005:~/tubes-abd$ docker compose up -d
[+] up 27/32
[+] up 27/32. [#####] 1.312GB / 1.809GB Pulling 24
[+] up 27/32. [#####] 1.313GB / 1.809GB Pulling 24
[+] up 27/32. [#####] 1.313GB / 1.809GB Pulling 24
[+] up 27/32. [#####] 1.313GB / 1.809GB Pulling 24
[+] up 27/32. [#####] 1.314GB / 1.809GB Pulling 24
[+] up 27/32. [#####] 1.315GB / 1.809GB Pulling 24
[+] up 27/32. [#####] 1.316GB / 1.809GB Pulling 24
[+] up 27/32. [#####] 1.316GB / 1.809GB Pulling 24
[+] up 27/32. [#####] 1.317GB / 1.809GB Pulling 24
[+] up 27/32. [#####] 1.317GB / 1.809GB Pulling 24
[+] up 27/32. [#####] 1.318GB / 1.809GB Pulling 24
[+] up 27/32. [#####] 1.318GB / 1.809GB Pulling 24
[+] up 27/32. [#####] 1.319GB / 1.809GB Pulling 24
[+] up 27/32. [#####] 1.32GB / 1.809GB Pulling 24
[+] up 27/32. [#####] 1.321GB / 1.809GB Pulling 24
[+] up 27/32. [#####] 1.321GB / 1.809GB Pulling 24
[+] up 27/32. [#####] 1.322GB / 1.809GB Pulling 24
[+] up 27/32. [#####] 1.323GB / 1.809GB Pulling 24
[+] up 27/32. [#####] 1.323GB / 1.809GB Pulling 24
[+] up 27/32. [#####] 1.323GB / 1.809GB Pulling 24
[+] up 27/32. [#####] 1.324GB / 1.809GB Pulling 24
[+] up 27/32. [#####] 1.325GB / 1.809GB Pulling 24
[+] up 27/32. [#####] 1.326GB / 1.809GB Pulling 24
[+] up 27/32. [#####] 1.327GB / 1.809GB Pulling 24
[+] up 27/32. [#####] 1.327GB / 1.809GB Pulling 24
[+] up 27/32. [#####] 1.327GB / 1.809GB Pulling 24
[+] up 27/32. [#####] 1.328GB / 1.809GB Pulling 24
[+] up 27/32. [#####] 1.329GB / 1.809GB Pulling 24
[+] up 27/32. [#####] 1.329GB / 1.809GB Pulling 24
[+] up 27/32. [#####] 1.329GB / 1.809GB Pulling 24
[+] up 27/32. [#####] 1.33GB / 1.809GB Pulling 24
[+] up 27/32. [#####] 1.331GB / 1.809GB Pulling 24
[+] up 28/32. [#####] 1.332GB / 1.809GB Pulling 24
[+] up 28/32. [#####] 1.333GB / 1.809GB Pulling 24
[+] up 28/32. [#####] 1.333GB / 1.809GB Pulling 24
[+] up 28/32. [#####] 1.334GB / 1.809GB Pulling 24
[+] up 28/32. [#####] 1.335GB / 1.809GB Pulling 24
[+] up 28/32. [#####] 1.335GB / 1.809GB Pulling 24
[+] up 28/32. [#####] 1.336GB / 1.809GB Pulling 24
```



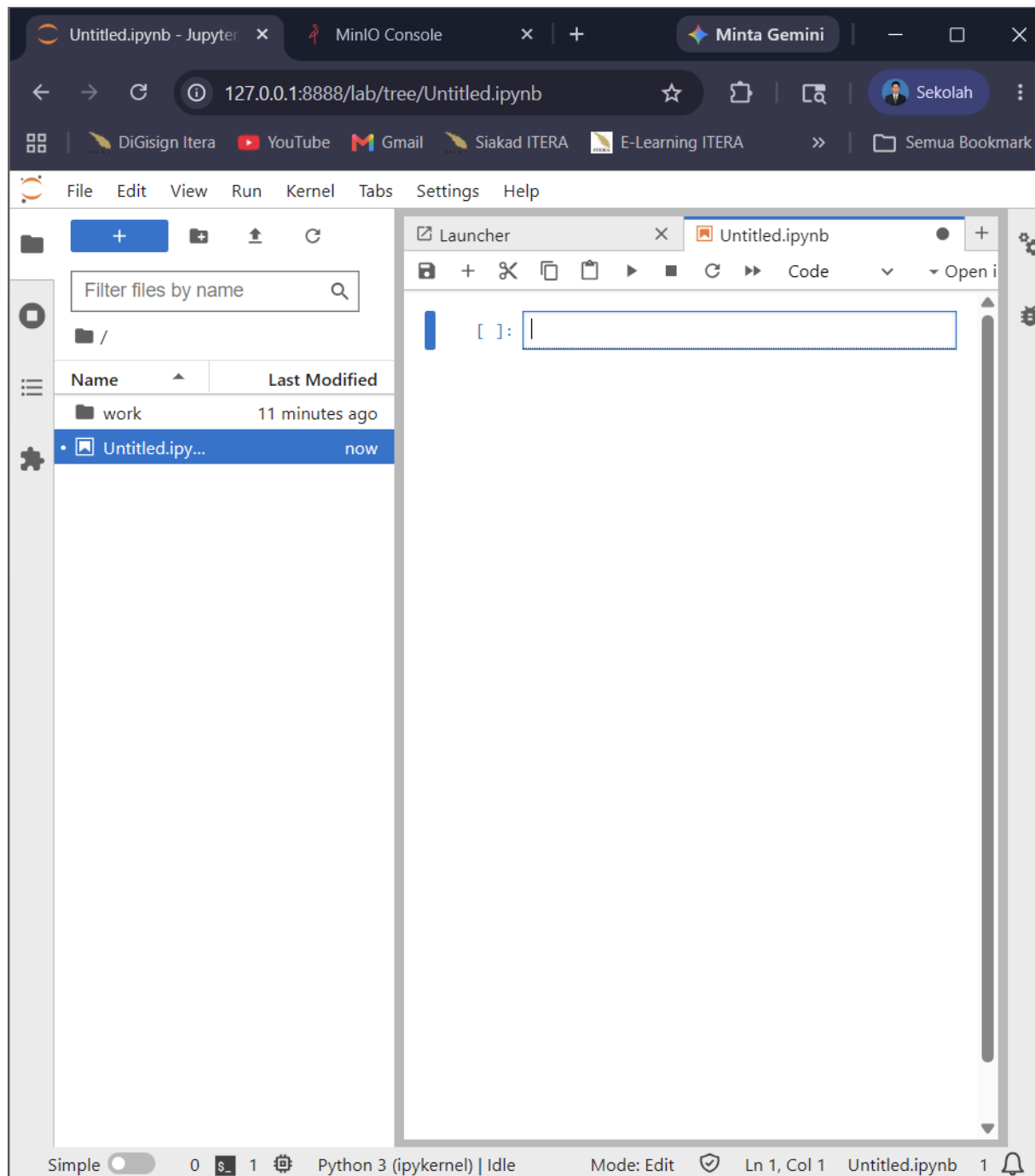
## Langkah 5: Mengakses JupyterLab dan Membuat Notebook

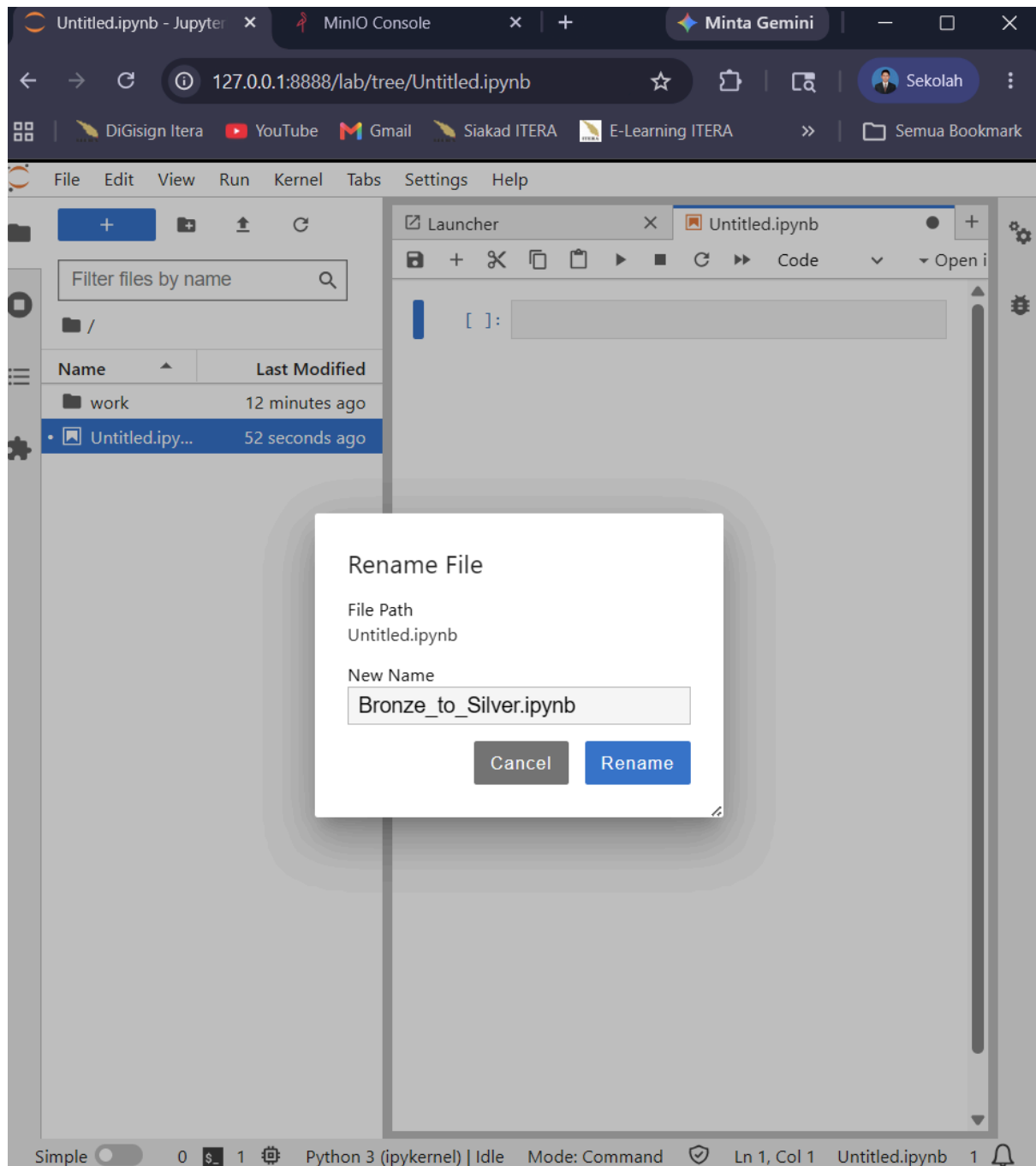


[http://127.0.0.1:8888/lab/tree/Bronze to Silver.ipynb](http://127.0.0.1:8888/lab/tree/Bronze%20to%20Silver.ipynb)

<http://127.0.0.1:8888/lab?token=4469f985361abbcecac42104d033aeebc982365c1f81d8f2>

## Langkah 6 : Proses Bronze ke Silver Layer





Launcher X Bronze\_to\_Silver.ipynb +

📁 + ✂ 📄 📌 ▶ ■ ↺ ⏩ Code ▾ ▾ Open i

```
[1]: !pip install openpyxl boto3 s3fs pandas

Requirement already satisfied: openpyxl in
/opt/conda/lib/python3.11/site-packages (3.
1.2)
Collecting boto3
  Downloading boto3-1.43.14-py3-none-any.wh
l.metadata (6.6 kB)
Collecting s3fs
  Downloading s3fs-2026.4.0-py3-none-any.wh
l.metadata (1.2 kB)
Requirement already satisfied: pandas in /o
pt/conda/lib/python3.11/site-packages (2.0.
3)
Requirement already satisfied: et-xmlfile i
n /opt/conda/lib/python3.11/site-packages
(from openpyxl) (1.1.0)
Collecting botocore<1.44.0,>=1.43.14 (from
boto3)
  Downloading botocore-1.43.14-py3-none-an
y.whl.metadata (5.6 kB)
Collecting jmespath<2.0.0,>=0.7.1 (from bot
o3)
  Downloading jmespath-1.1.0-py3-none-any.w
hl.metadata (7.6 kB)
Collecting s3transfer<0.18.0,>=0.17.0 (from
boto3)
  Downloading s3transfer-0.17.0-py3-none-an
y.whl.metadata (1.7 kB)
Collecting aiobotocore<4.0.0,>=2.19.0 (from
s3fs)
  Downloading aiobotocore-3.7.0-py3-none-an
y.whl.metadata (28 kB)
Collecting fsspec==2026.4.0 (from s3fs)
  Downloading fsspec-2026.4.0-py3-none-any.
whl.metadata (10 kB)
```

kernel) |... Mode: Comm... 🔒 Ln 1, Col... Bronze\_to\_Silver.ip... 1



Launcher X Bronze\_to\_Silver.ipynb +

📁 + ✂ 📄 📌 ▶ ■ ↺ ▶▶ Code ▼ Open i

```
Downloading s3fs-0.6.0-py3-none-any.whl.metadata (5.2 kB)
Collecting s3fs
  Downloading s3fs-0.6.0-py3-none-any.whl.metadata (1.4 kB)
  Downloading s3fs-0.5.2-py3-none-any.whl.metadata (1.2 kB)
  Downloading s3fs-0.5.1-py3-none-any.whl.metadata (1.3 kB)
  Downloading s3fs-0.5.0-py3-none-any.whl.metadata (1.3 kB)
  Downloading s3fs-0.4.2-py3-none-any.whl.metadata (1.3 kB)
Requirement already satisfied: six>=1.5 in /opt/conda/lib/python3.11/site-packages (from python-dateutil>=2.8.2->pandas) (1.16.0)
Downloading boto3-1.43.14-py3-none-any.whl (140 kB)

140.5/140.5 kB 8.6 MB/s eta 0:00:00
Downloading botocore-1.43.14-py3-none-any.whl (15.1 MB)

15.1/15.1 MB 4.5 MB/s eta 0:00:00
Downloading s3fs-0.4.2-py3-none-any.whl (19 kB)
Downloading jmespath-1.1.0-py3-none-any.whl (20 kB)
Downloading s3transfer-0.17.0-py3-none-any.whl (86 kB)

86.8/86.8 kB 3.6 MB/s eta 0:00:00
Installing collected packages: jmespath, botocore, s3transfer, s3fs, boto3
Successfully installed boto3-1.43.14 botocore-1.43.14 jmespath-1.1.0 s3fs-0.4.2 s3transfer-0.17.0
```

/kernel) |... Mode: Comm... ✓ Ln 1, Col... Bronze\_to\_Silver.ip... 1

```

import pandas as pd
import boto3
from pyspark.sql import SparkSession
from io import BytesIO

# 1. Inisialisasi Apache Spark Session
spark = SparkSession.builder \
    .appName("SDG_Goal8_BronzeToSilver") \
    .getOrCreate()
print("Spark Session berhasil dibuat!")

# 2. Mengambil Raw Data dari MinIO (Bronze Layer)
s3 = boto3.client(
    's3',
    endpoint_url='http://minio:9000',
    aws_access_key_id='admin',
    aws_secret_access_key='admin123'
)

print("Mengunduh file Goal8.xlsx dari Bronze Layer...")
obj = s3.get_object(Bucket='bronze', Key='Goal8.xlsx')
excel_data = obj['Body'].read()

# Membaca Excel menggunakan Pandas (karena Spark tidak memiliki native reader untuk Excel)
pdf = pd.read_excel(BytesIO(excel_data), engine='openpyxl')

# 3. Proses Pembersihan Data (Data Cleaning)
# Menghapus baris yang sepenuhnya kosong
pdf_clean = pdf.dropna(how='all')

# Menstandarkan nama kolom (menghapus spasi dan karakter khusus agar kompatibel dengan Spark/Parquet)
pdf_clean.columns = [c.replace(' ', '_').replace('(', '').replace(')', '').replace('.', '') for c in pdf_clean.columns]

# Konversi ke Spark DataFrame untuk melihat skemanya
df_spark = spark.createDataFrame(pdf_clean)
print("Data berhasil dibersihkan! Menampilkan 5 baris pertama data di Spark:")
df_spark.show(5)

# 4. Menyimpan Hasil ke MinIO (Silver Layer) dalam format Parquet
print("Menyimpan data bersih ke Silver Layer...")
pdf_clean.to_parquet(
    's3://silver/Goal8_clean.parquet',
    storage_options={

```



```

        "key": "admin",
        "secret": "admin123",
        "client_kwargs": {"endpoint_url": "http://minio:9000"}
    },
    index=False
)
print("PROSES SELESAI! Data sudah tersimpan di bucket Silver sebagai
Goal8_clean.parquet.")

```

The screenshot shows a Jupyter Notebook window titled "Bronze\_to\_Silver.ipynb". The code is as follows:

```

[*]: import pandas as pd
import boto3
from pyspark.sql import SparkSession
from io import BytesIO

# 1. Inisialisasi Apache Spark Session
spark = SparkSession.builder \
    .appName("SDG_Goal8_BronzeToSilver") \
    .getOrCreate()
print("Spark Session berhasil dibuat!")

# 2. Mengambil Raw Data dari MinIO (Bronze Layer)
s3 = boto3.client(
    's3',
    endpoint_url='http://minio:9000',
    aws_access_key_id='admin',
    aws_secret_access_key='admin123'
)

print("📄 Mengunduh file Goal8.xlsx dari Bronze Layer...")
obj = s3.get_object(Bucket='bronze', Key='Goal8.xlsx')
excel_data = obj['Body'].read()

# Membaca Excel menggunakan Pandas (karena Spark tidak memiliki native reader untuk Excel)
pdf = pd.read_excel(BytesIO(excel_data), engine='openpyxl')

# 3. Proses Pembersihan Data (Data Cleaning)
# Menghapus baris yang sepenuhnya kosong
pdf_clean = pdf.dropna(how='all')

# Menstandarkan nama kolom (menghapus spasi dan karakter khusus agar kompatibel dengan Spark/Parquet)
pdf_clean.columns = [c.replace(' ', '_').replace('(', '').replace(')', '').replace('.', '') for c in pdf_clean.columns]

# Konversi ke Spark DataFrame untuk melihat skemanya

```

The bottom status bar of the Jupyter Notebook shows "pykernel | Busy" on the left and "Mode: Command" with a shield icon on the right.

```

print("📄 Mengunduh file Goal8.xlsx dari Bronze Layer...")
obj = s3.get_object(Bucket='bronze', Key='Goal8.xlsx')
excel_data = obj['Body'].read()

# Membaca Excel menggunakan Pandas (karena Spark tidak memiliki native reader untuk Excel)
pdf = pd.read_excel(BytesIO(excel_data), engine='openpyxl')

# 3. Proses Pembersihan Data (Data Cleaning)
# Menghapus baris yang sepenuhnya kosong
pdf_clean = pdf.dropna(how='all')

# Menstandarkan nama kolom (menghapus spasi dan karakter khusus agar kompatibel dengan Spark/Parquet)
pdf_clean.columns = [c.replace(' ', '_').replace('(', '').replace(')', '').replace('.', '') for c in pdf_clean.columns]

# Konversi ke Spark DataFrame untuk melihat skemanya
df_spark = spark.createDataFrame(pdf_clean)
print("Data berhasil dibersihkan! Menampilkan 5 baris pertama data di Spark:")
df_spark.show(5)

# 4. Menyimpan Hasil ke MinIO (Silver Layer) dalam format Parquet
print("Menyimpan data bersih ke Silver Layer...")
pdf_clean.to_parquet(
    's3://silver/Goal8_clean.parquet',
    storage_options={
        "key": "admin",
        "secret": "admin123",
        "client_kwargs": {"endpoint_url": "http://minio:9000"}
    },
    index=False
)
print("PROSES SELESAI! Data sudah tersimpan di bucket Silver sebagai Goal8_clean.parquet.")

Spark Session berhasil dibuat!
📄 Mengunduh file Goal8.xlsx dari Bronze Layer...

```

The screenshot shows a Jupyter Notebook window titled "Bronze\_to\_Silver.ipynb". The notebook contains two code cells. The first cell has Python code that sets `index=False` and prints a message about saving data as a Parquet file. The second cell displays the output of a Spark session, which includes a message about downloading a file from Bronze Layer and displaying the top rows of data. The data is presented as a wide table with columns for various indicators and their values over time. The bottom status bar indicates the user is idle at 10:47 AM on 8/6/2024.

```
}},  
    index=False  
)  
  
print("PROSES SELESAI! Data sudah tersimpan di bucket Silver sebagai Goal8_clean.parquet.")
```

Spark Session berhasil dibuat!  
Mengunduh file Goal8.xlsx dari Bronze Layer...  
Data berhasil dibersihkan! Menampilkan 5 baris pertama data di Spark:

| [GoalId]                 | [TargetIndicator] | [SeriesCode]                            | [SeriesDescription] | [GeoAreaCode] | [GeoAreaName] | [TimePeriod] | [Value] | [Time_Detail] | [TimeCoverage] | [UpperBound] | [LowerBound] | [BasePeriod] |
|--------------------------|-------------------|-----------------------------------------|---------------------|---------------|---------------|--------------|---------|---------------|----------------|--------------|--------------|--------------|
| 0.0 United Nations, D... | G Na N            | 8.1.1 NY_GDP_PCAP Annual growth rate... | rat...              | 4 Afghanistan | 2000 -6.6     | 2000         | NaN     | CA            | A              | 202          | Na           |              |
| 0.0 United Nations, D... | G Na N            | 8.1.1 NY_GDP_PCAP Annual growth rate... | rat...              | 4 Afghanistan | 2001 -6.4     | 2001         | NaN     | CA            | A              | 202          | Na           |              |
| 0.0 United Nations, D... | G Na N            | 8.1.1 NY_GDP_PCAP Annual growth rate... | rat...              | 4 Afghanistan | 2002 56.6     | 2002         | NaN     | CA            | A              | 202          | Na           |              |
| 0.0 United Nations, D... | G Na N            | 8.1.1 NY_GDP_PCAP Annual growth rate... | rat...              | 4 Afghanistan | 2003 1.0      | 2003         | NaN     | CA            | A              | 202          | Na           |              |
| 0.0 United Nations, D... | G Na N            | 8.1.1 NY_GDP_PCAP Annual growth rate... | rat...              | 4 Afghanistan | 2004 -2.0     | 2004         | NaN     | CA            | A              | 202          | Na           |              |

only showing top 5 rows

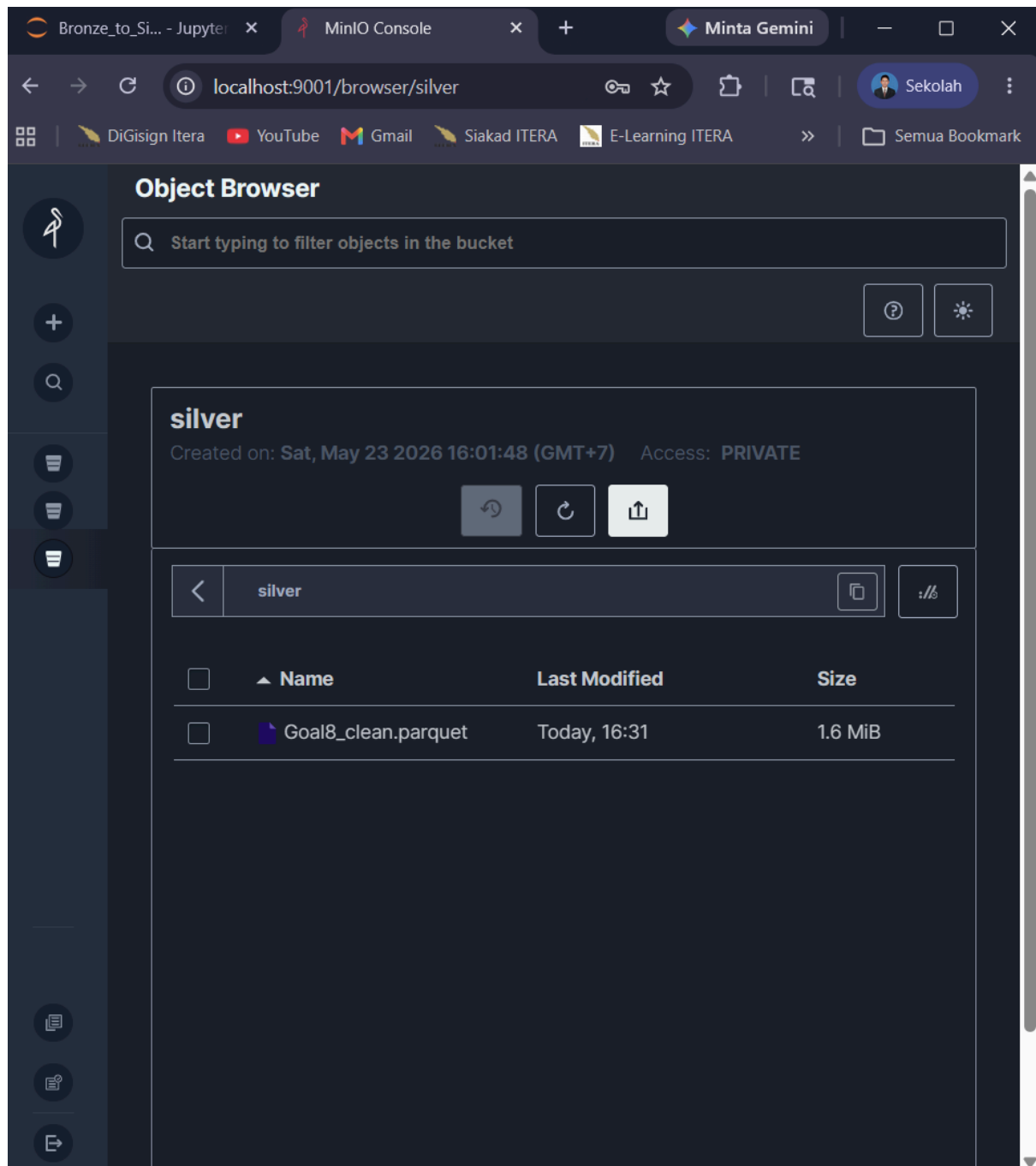
```
Goal|Target|Indicator| SeriesCode| SeriesDescription|GeoAreaCode|GeoAreaName|TimePeriod|Value|Time_Detail|TimeCoverage|UpperBound|LowerBound|BasePer
iod| Source|GeoInfotrl|FootNote|Activity|Age|Disability_status|Education_level|Location|Migratory_status|Nature|Observation_Status|Quantil
e|Reporting_Type|Sex|Type_of_occupation|Type_of_product| Units|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 8| 8.1| 8.1.1|NY_GDP_PCAP|Annual growth rat...| 4|Afghanistan| 2000| -6.6| 2000| NaN| NaN| CA| NaN| A| 202
0.0|United Nations, D...| NaN| NaN| NaN|NaN| NaN| NaN| NaN| NaN| NaN| CA| A| Na
N| G|NaN| NaN| NaN|PERCENT|
| 8| 8.1| 8.1.1|NY_GDP_PCAP|Annual growth rat...| 4|Afghanistan| 2001| -6.4| 2001| NaN| NaN| CA| NaN| A| 202
0.0|United Nations, D...| NaN| NaN| NaN|NaN| NaN| NaN| NaN| NaN| NaN| CA| A| Na
N| G|NaN| NaN| NaN|PERCENT|
| 8| 8.1| 8.1.1|NY_GDP_PCAP|Annual growth rat...| 4|Afghanistan| 2002| 56.6| 2002| NaN| NaN| CA| NaN| A| 202
0.0|United Nations, D...| NaN| NaN| NaN|NaN| NaN| NaN| NaN| NaN| NaN| CA| A| Na
N| G|NaN| NaN| NaN|PERCENT|
| 8| 8.1| 8.1.1|NY_GDP_PCAP|Annual growth rat...| 4|Afghanistan| 2003| 1.0| 2003| NaN| NaN| CA| NaN| A| 202
0.0|United Nations, D...| NaN| NaN| NaN|NaN| NaN| NaN| NaN| NaN| NaN| CA| A| Na
N| G|NaN| NaN| NaN|PERCENT|
| 8| 8.1| 8.1.1|NY_GDP_PCAP|Annual growth rat...| 4|Afghanistan| 2004| -2.0| 2004| NaN| NaN| CA| NaN| A| 202
0.0|United Nations, D...| NaN| NaN| NaN|NaN| NaN| NaN| NaN| NaN| NaN| CA| A| Na
N| G|NaN| NaN| NaN|PERCENT|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows

Menyimpan data bersih ke Silver Layer...

/opt/conda/lib/python3.11/site-packages/fsspec/registry.py:271: UserWarning: Your installed version of s3fs is very old and known to cause
severe performance issues, see also https://github.com/dask/dask/issues/10276

To fix, you should specify a lower version bound on s3fs, or
update the current installation.

warnings.warn(s3_msg)
PROSES SELESAI! Data sudah tersimpan di bucket Silver sebagai Goal8_clean.parquet.
```



Kami menulis dan menjalankan kode Python di notebook `Bronze_to_Silver.ipynb` yang mencakup empat proses utama: (1) menginisialisasi Apache Spark Session dengan nama aplikasi `SDG_Goal8_BronzeToSilver`, (2) mengunduh file `Goal8.xlsx` dari bucket `bronze` di MinIO menggunakan `boto3`, (3) melakukan data cleaning menggunakan `Pandas` dengan menghapus baris kosong dan menstandarkan nama kolom agar kompatibel dengan format `Parquet`, serta (4) mengonversi data ke `Spark DataFrame` dan menyimpan hasilnya ke bucket `silver` dalam format `Goal8_clean.parquet`. Proses ini berhasil dijalankan dan kelima baris pertama data berhasil ditampilkan melalui `Spark`.

## Langkah 6: Proses Silver ke Gold Layer

```
import pandas as pd

# 1. Mengambil data bersih dari Silver Layer
print("Mengambil data dari Silver Layer...")
pdf_silver = pd.read_parquet(
    's3://silver/Goal8_clean.parquet',
    storage_options={
        "key": "admin",
        "secret": "admin123",
        "client_kwargs": {"endpoint_url": "http://minio:9000"}
    }
)

# 2. Mengubah data menjadi Spark DataFrame untuk pemrosesan terdistribusi
df_spark_silver = spark.createDataFrame(pdf_silver)

# Menampilkan kolom apa saja yang tersedia agar Anda bisa merancang analisis
print("Skema Data Silver:")
df_spark_silver.printSchema()

# 3. Membuat Temporary View agar bisa diproses menggunakan Spark SQL
df_spark_silver.createOrReplaceTempView("tabel_sdg8")

print("Melakukan Agregasi Data (Transformasi Gold Layer)...")
#
=====
=====
# BAGIAN INI BISA ANDA UBAH SESUAI KEBUTUHAN ANALISIS PROPOSAL
# ANDA
# Sebagai contoh: Kita menghitung jumlah baris data per entitas/tahun
# Ganti nama kolom sesuai dengan hasil printSchema() di atas!
#
=====
=====
query_gold = """
    SELECT
        * FROM tabel_sdg8
    LIMIT 100
"""

# Catatan: Ubah query di atas. Contoh agregasi:
# SELECT GeoAreaName, AVG(Value) as Rata_Rata_Pertumbuhan FROM tabel_sdg8
# GROUP BY GeoAreaName

# Mengeksekusi query Spark SQL
df_gold = spark.sql(query_gold)
print("Hasil Agregasi Gold Layer (5 Baris Pertama):")
```

```
df_gold.show(5)
```

#### # 4. Menyimpan Hasil Agregasi ke MinIO (Gold Layer)

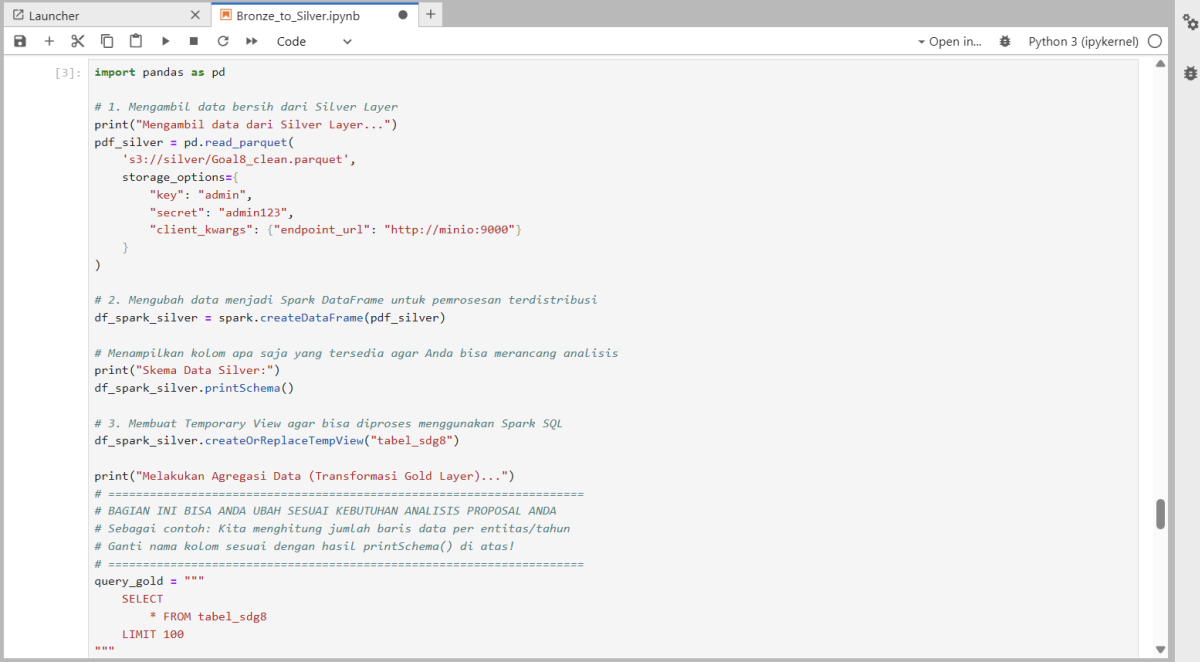
```
print("Menyimpan data agregasi ke Gold Layer...")
```

```
# Konversi kembali ke Pandas untuk mempermudah upload S3
```

```
pdf_gold = df_gold.toPandas()
```

```
pdf_gold.to_parquet(  
    's3://gold/Goal8_aggregated.parquet',  
    storage_options={  
        "key": "admin",  
        "secret": "admin123",  
        "client_kwargs": {"endpoint_url": "http://minio:9000"}  
    },  
    index=False  
)
```

```
print("PROSES MEDALLION SELESAI! Data agregasi analitik sudah tersimpan di bucket Gold.")
```



```
[3]: import pandas as pd  
  
# 1. Mengambil data bersih dari Silver Layer  
print("Mengambil data dari Silver Layer...")  
pdf_silver = pd.read_parquet(  
    's3://silver/Goal8_clean.parquet',  
    storage_options={  
        "key": "admin",  
        "secret": "admin123",  
        "client_kwargs": {"endpoint_url": "http://minio:9000"}  
    }  
)  
  
# 2. Mengubah data menjadi Spark DataFrame untuk pemrosesan terdistribusi  
df_spark_silver = spark.createDataFrame(pdf_silver)  
  
# Menampilkan kolom apa saja yang tersedia agar Anda bisa merancang analisis  
print("Skema Data Silver:")  
df_spark_silver.printSchema()  
  
# 3. Membuat Temporary View agar bisa diproses menggunakan Spark SQL  
df_spark_silver.createOrReplaceTempView("tabel_sdgs")  
  
print("Melakukan Agregasi Data (Transformasi Gold Layer)...")  
# =====  
# BAGIAN INI BISA ANDA UBAH SESUAI KEBUTUHAN ANALISIS PROPOSAL ANDA  
# Sebagai contoh: Kita menghitung jumlah baris data per entitas/tahun  
# Ganti nama kolom sesuai dengan hasil printSchema() di atas!  
# =====  
query_gold = """  
    SELECT  
        * FROM tabel_sdgs  
    LIMIT 100  
    """
```

ipykernel | Idle Mode: Command Ln 1, Col 1 Bronze\_to\_Silver.ipynb 1

```
Launcher x Bronze_to_Silver.ipynb +
LIMIT 100

"""
# Catatan: Ubah query di atas. Contoh agregasi:
# SELECT GeoAreaName, AVG(Value) as Rata_Rata_Pertumbuhan FROM tabel_sdg8 GROUP BY GeoAreaName

# Meneksekusi query Spark SQL
df_gold = spark.sql(query_gold)
print("Hasil Agregasi Gold Layer (5 Baris Pertama):")
df_gold.show(5)

# 4. Menyimpan Hasil Agregasi ke MinIO (Gold Layer)
print("Menyimpan data agregasi ke Gold Layer...")
# Konversi kembali ke Pandas untuk mempermudah upload S3
pdf_gold = df_gold.toPandas()

pdf_gold.to_parquet(
    's3://gold/Goal8_aggregated.parquet',
    storage_options={
        "key": "admin",
        "secret": "admin123",
        "client_kwargs": {"endpoint_url": "http://minio:9000"}
    },
    index=False
)
print("PROSES MEDALLION SELESAIL! Data agregasi analitik sudah tersimpan di bucket Gold.")

Mengambil data dari Silver Layer...
Skema Data Silver:
root
|-- Goal: long (nullable = true)
|-- Target: string (nullable = true)
|-- Indicator: string (nullable = true)
|-- SeriesCode: string (nullable = true)
|-- SeriesDescription: string (nullable = true)
|-- GeoAreaCode: long (nullable = true)
|-- GeoAreaName: string (nullable = true)
|-- TimePeriod: long (nullable = true)
|-- Value: double (nullable = true)
|-- Time_Detail: string (nullable = true)
|-- TimeCoverage: double (nullable = true)
|-- UpperBound: double (nullable = true)
|-- LowerBound: double (nullable = true)
|-- BasePeriod: double (nullable = true)
|-- Source: string (nullable = true)
|-- GeoInfoUrl: double (nullable = true)
|-- FootNote: string (nullable = true)
|-- Activity: string (nullable = true)
|-- Age: string (nullable = true)
|-- Disability_status: string (nullable = true)
|-- Education_level: string (nullable = true)
|-- Location: string (nullable = true)
|-- Migratory_status: string (nullable = true)
|-- Nature: string (nullable = true)
|-- Observation_Status: string (nullable = true)
|-- Quantile: string (nullable = true)
|-- Reporting_Type: string (nullable = true)
|-- Sex: string (nullable = true)
|-- Type_of_occupation: string (nullable = true)
|-- Type_of_product: string (nullable = true)
|-- Units: string (nullable = true)

ipykernel | Idle Mode: Edit Ln 14, Col 74 Bronze_to_Silver.ipynb 1
```

```
Launcher x Bronze_to_Silver.ipynb +

SeriesDescription: string (nullable = true)
GeoAreaCode: long (nullable = true)
GeoAreaName: string (nullable = true)
TimePeriod: long (nullable = true)
Value: double (nullable = true)
Time_Detail: string (nullable = true)
TimeCoverage: double (nullable = true)
UpperBound: double (nullable = true)
LowerBound: double (nullable = true)
BasePeriod: double (nullable = true)
Source: string (nullable = true)
GeoInfoUrl: double (nullable = true)
FootNote: string (nullable = true)
Activity: string (nullable = true)
Age: string (nullable = true)
Disability_status: string (nullable = true)
Education_level: string (nullable = true)
Location: string (nullable = true)
Migratory_status: string (nullable = true)
Nature: string (nullable = true)
Observation_Status: string (nullable = true)
Quantile: string (nullable = true)
Reporting_Type: string (nullable = true)
Sex: string (nullable = true)
Type_of_occupation: string (nullable = true)
Type_of_product: string (nullable = true)
Units: string (nullable = true)

Melakukan Agregasi Data (Transformasi Gold Layer)...
Hasil Agregasi Gold Layer (5 Baris Pertama):

+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|Goal|Target|Indicator|SeriesCode|SeriesDescription|GeoAreaCode|GeoAreaName|TimePeriod|Value|Time_Detail|TimeCoverage|UpperBound|LowerBound|BasePer|
|iod|Source|GeoInfoUrl|FootNote|Activity|Age|Disability_status|Education_level|Location|Migratory_status|Nature|Observation_Status|Quantile|Reporting_Type|Sex|Type_of_occupation|Type_of_product|Units|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

ipykernel | Idle Mode: Edit Ln 14, Col 74 Bronze_to_Silver.ipynb 1
```

Launcher

Bronze\_to\_Silver.ipynb

Python 3 (ipykernel)

Code

```
-- type_of_occupation: string (nullable = true)
-- Type_of_product: string (nullable = true)
-- Units: string (nullable = true)

Melakukan Agregasi Data (Transformasi Gold Layer)...
Hasil Agregasi Gold Layer (5 Baris Pertama):
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|Goal|Target|Indicator|SeriesCode|SeriesDescription|GeoAreaCode|GeoAreaName|TimePeriod|Value|Time_Detail|TimeCoverage|UpperBound|LowerBound|BasePer
id|Source|GeoInfoUrl|FootNote|Activity|Age|Disability_status|Education_level|Location|Migratory_status|Nature|Observation_Status|Quant
le|Reporting_Type|Sex|Type_of_occupation|Type_of_product|Units|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 8| 8.1| 8.1.1|NY_GDP_PCAP|Annual growth rat...| 4|Afghanistan| 2000| -6.6| 2000| NaN| NaN| NaN| 202
0.0|United Nations, D...| NaN| NULL| NULL|NULL| NULL| NULL| NULL| NULL| CA| A| NU
LL| G|NULL| NULL| NULL|PERCENT|
| 8| 8.1| 8.1.1|NY_GDP_PCAP|Annual growth rat...| 4|Afghanistan| 2001| -6.4| 2001| NaN| NaN| NaN| 202
0.0|United Nations, D...| NaN| NULL| NULL|NULL| NULL| NULL| NULL| NULL| CA| A| NU
LL| G|NULL| NULL| NULL|PERCENT|
| 8| 8.1| 8.1.1|NY_GDP_PCAP|Annual growth rat...| 4|Afghanistan| 2002| 56.6| 2002| NaN| NaN| NaN| 202
0.0|United Nations, D...| NaN| NULL| NULL|NULL| NULL| NULL| NULL| NULL| CA| A| NU
LL| G|NULL| NULL| NULL|PERCENT|
| 8| 8.1| 8.1.1|NY_GDP_PCAP|Annual growth rat...| 4|Afghanistan| 2003| 1.0| 2003| NaN| NaN| NaN| 202
0.0|United Nations, D...| NaN| NULL| NULL|NULL| NULL| NULL| NULL| NULL| CA| A| NU
LL| G|NULL| NULL| NULL|PERCENT|
| 8| 8.1| 8.1.1|NY_GDP_PCAP|Annual growth rat...| 4|Afghanistan| 2004| -2.0| 2004| NaN| NaN| NaN| 202
0.0|United Nations, D...| NaN| NULL| NULL|NULL| NULL| NULL| NULL| NULL| CA| A| NU
LL| G|NULL| NULL| NULL|PERCENT|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows

Menyimpan data agregasi ke Gold Layer...
PROSES MEDALLION SELESAI! Data agregasi analitik sudah tersimpan di bucket Gold.
```

ipykernel) | Idle

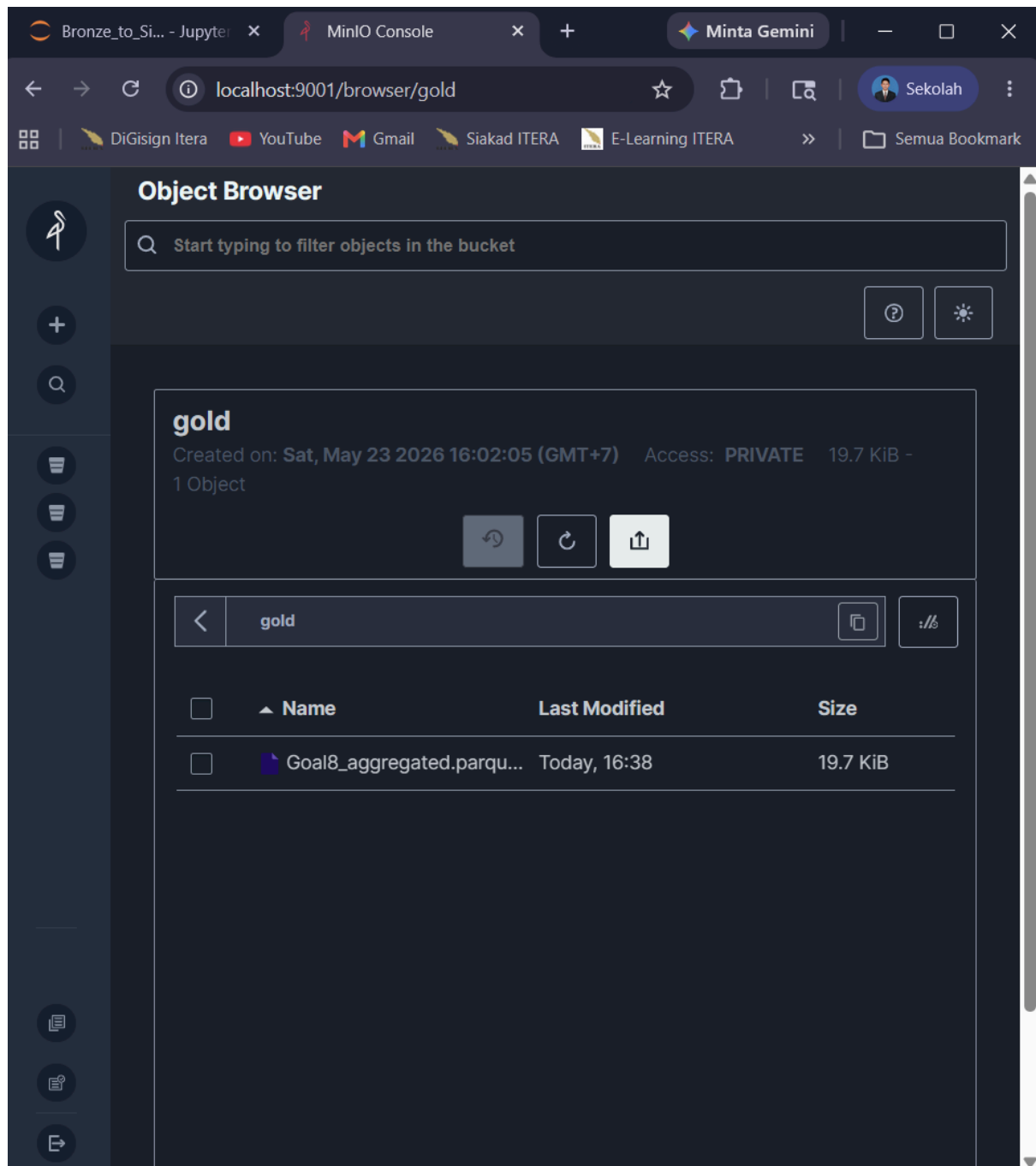
Mode: Edit

Ln 14, Col 74

Bronze\_to\_Silver.ipynb

1





Dengan selesainya seluruh langkah di atas, kelompok kami berhasil membangun pipeline Arsitektur Medallion secara end-to-end menggunakan Apache Spark dan MinIO. Pipeline ini mencakup alur data dari Bronze (raw data) → Silver (cleaned data) → Gold (aggregated analytics), yang siap digunakan sebagai dasar visualisasi dan analisis deskriptif-asosiatif terkait SDG Goal 8.